

Internet Of Things

Projeto Final

Rodrigo Henriques Nº 25268

Gonçalo Henriques Nº 21590

Índice

Introdução 3

The data collected by the sensors and the actuators must be available from anywhere, despite using private IPv4 addresses. 4

Autenticação e Proteção contra Acesso Não Autorizado 6

Modo Diurno/Noturno e Feedback Sonoro (LDR + LED + Buzzer) 7

Alteração automática do tema com base na luminosidade (modo claro/escuro) 8

..... 9

Calibração do Sensor LDR 9

Implementação de Funcionalidades MQTT: Keep Alive, Last Will e Retained Message 10

Integração com API Externa para Notificações em Tempo Real (ntfy.sh) 12

The solution should also include a provisioning mechanism to add, remove, and replace devices. 16

Conclusão 17

Introdução

A nossa solução para este projeto consiste no desenvolvimento de uma câmara de vigilância inteligente baseada em tecnologias de baixo custo e baixo consumo energético. A proposta visa responder a um problema real: a necessidade de sistemas de monitorização autónomos, acessíveis e fáceis de configurar, especialmente em contextos residenciais, educativos ou pequenos espaços comerciais.

Para tal, foi implementado um sistema de câmara IoT utilizando um Raspberry Pi Pico W, que comunica via protocolo MQTT com uma interface web e envia notificações em tempo real. O dispositivo é capaz de detetar movimento, alternar automaticamente entre modo diurno e noturno com base na luz ambiente que é feita a partir do LDR, e emitir alertas sonoros e visuais conforme o estado do sistema.

O sistema é composto por sensores e atuadores externos, nomeadamente:

- Um sensor PIR para deteção de movimento;
- Um sensor LDR para medição de luz ambiente;
- Um LED RGB e um buzzer como indicadores de estado.

Além disso, foi desenvolvido um mecanismo de configuração Wi-Fi automático (“provisioning”), que permite adicionar ou trocar de rede diretamente através de um portal local (sem necessidade de cabos ou interfaces adicionais). Este portal é ativado sempre que o dispositivo não consegue ligar-se a uma rede guardada previamente.

A solução destaca-se por ser portátil, autónoma e de fácil adaptação a diferentes contextos de utilização, oferecendo funcionalidades relevantes com um investimento mínimo em hardware.

The data collected by the sensors and the actuators must be available from anywhere, despite using private IPv4 addresses.

Para garantir que os dados recolhidos pelos sensores (como o movimento e a luminosidade) e os comandos enviados para os atuadores (LED RGB, buzzer) estivessem acessíveis a partir de qualquer local, mesmo quando o dispositivo se encontra numa rede com endereços IPv4 privados, foi implementado um sistema de comunicação baseado no protocolo MQTT em conjunto com a ferramenta ngrok.

O Raspberry Pi Pico W comunica com um broker MQTT que corre localmente numa máquina na mesma rede (com endereço privado, como 192.168.x.x). Para permitir o acesso remoto, o broker é exposto à Internet através de um túnel seguro criado com o ngrok, que gera automaticamente um endereço público do tipo wss://..., acessível a partir de qualquer local.

A interface web de controlo, servida localmente (através do comando py -m http.server 8080) ou exposta também via ngrok, liga-se ao broker MQTT via WebSockets. Esta interface permite ao utilizador:

- Consultar o estado da câmara e dos sensores em tempo real;
- Ligar ou desligar a câmara (LED);
- Ativar ou silenciar o som (buzzer);
- Receber alertas visuais e sonoros sempre que o sistema deteta movimento ou muda de estado.

Apesar de o Raspberry Pi Pico W permanecer na rede local e comunicar com o broker através do IP privado da máquina principal, o uso do ngrok torna os serviços acessíveis globalmente. Esta abordagem cumpre integralmente o requisito funcional de permitir acesso externo aos dados dos sensores e aos atuadores, sem necessidade de IP público, reencaminhamento de portas ou configurações avançadas no router.

Nota: a versão gratuita do ngrok gera novos links públicos a cada reinício. Por isso, é necessário atualizar o ficheiro index_mqtt.html com o novo endereço do broker MQTT (wss://...) sempre que o ngrok é relançado.

```
version: "3"
agent:
  authtoken: 2ulpYYvo3QhTrKYynRnSquuGb23_VxSQeuak24G3eQC7YFjJ

tunnels:
  web:
    proto: http
    addr: 8080
  mqtt:
    proto: http
    addr: 9001
```

Figura 1 - Ficheiro ngrok.yml

```
Take our ngrok in production survey! https://forms.gle/aXiBFWzEA36DudFn6

Session Status                online
Account                       coisas.s.silva@gmail.com (Plan: Free)
Update                        update available (version 3.22.1, Ctrl-U to update)
Version                       3.22.0
Region                        Europe (eu)
Latency                       142ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://4dbe-89-214-155-121.ngrok-free.app -> http://localhost:9001
Forwarding                    https://d826-89-214-155-121.ngrok-free.app -> http://localhost:8080

Connections                   ttl     opn     rt1     rt5     p50     p90
                              10      1       0.00    0.01    0.01    151.90

HTTP Requests
-------------

15:10:03.795 BST GET /favicon.ico              404 File not found
15:10:03.913 BST GET /                         101 Switching Protocols
15:10:03.142 BST GET /index_mqtt.html          200 OK
15:08:45.922 BST GET /                         101 Switching Protocols
15:08:45.276 BST GET /index_mqtt.html          200 OK
15:08:45.755 BST GET /favicon.ico              404 File not found
15:07:56.912 BST GET /                         101 Switching Protocols
15:07:56.769 BST GET /favicon.ico              404 File not found
15:07:55.598 BST GET /index_mqtt.html          200 OK
15:06:03.128 BST GET /                         502 Bad Gateway
```

Figura 2 - Links públicos da Interface Web e do Broker MQTT (WebSocket)

Autenticação e Proteção contra Acesso Não Autorizado

Para garantir que apenas os utilizadores autorizados possam aceder ao controlo da câmara, foi implementado um sistema de autenticação via interface web. Esta autenticação é realizada através de uma página de login (login.html), onde o utilizador introduz o seu nome de utilizador e palavra-passe. As credenciais são validadas no lado do cliente (JavaScript), e apenas após um login bem-sucedido é que o utilizador tem acesso à interface principal (index_mqtt_login.html), na qual comunica com o broker MQTT via WebSockets.

A interface inclui ainda um botão de logout, que termina a sessão e impede que o utilizador volte a aceder à interface sem se autenticar novamente, mesmo que tente aceder diretamente pelo URL. Esta abordagem assegura que o controlo da câmara só pode ser feito por quem tem credenciais válidas, cumprindo assim os requisitos de segurança e controlo de acesso definidos no projeto.

Para testar o sistema de permissões, foi criado um segundo utilizador chamado "convidado", com acesso restrito podendo fazer login e visualizar a interface, mas não tem permissões para controlar a câmara (ex: ligar/desligar LED).

Foram feitos alguns procedimentos tais como:

1. No terminal (cmd), com permissões de administrador, foi criado o utilizador com o comando "mosquitto_passwd -b passwords.txt convidado 1234".
2. No ficheiro de permissões (aclfile.txt), foi adicionado: user convidado
3. No ficheiro login.html, foi adicionada a validação com o

   if ((user === "teste" && pass === "123") || (user === "convidado" && pass === "1234")) {

4. Por último reiniciou-se o serviço Mosquitto para aplicar as alterações.

Como resultado, foi verificado que o utilizador "convidado" consegue efetuar login e aceder à interface, mas não consegue controlar os atuadores nem enviar mensagens via MQTT pois o broker recusa as suas tentativas de publicação/subscrição. Isto mostra que o sistema está corretamente protegido contra acessos não autorizados a funcionalidades críticas, cumprindo os requisitos de autenticação e segurança remota.

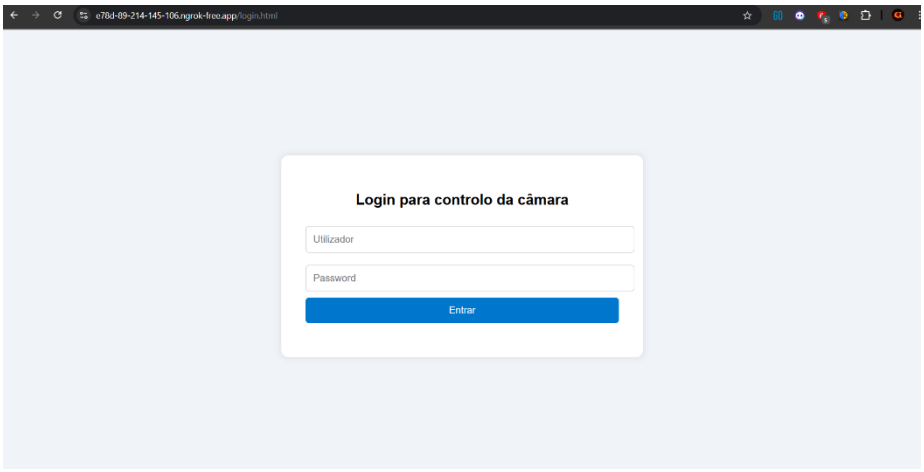


Figura 3 - Login

Modo Diurno/Noturno e Feedback Sonoro (LDR + LED + Buzzer)

Com o objetivo de simular o comportamento realista de uma câmara de vigilância, foi integrado no sistema um sensor de luminosidade (LDR) ligado ao pino GP28 do Raspberry Pi Pico W. Este sensor permite detetar a presença ou ausência de luz ambiente, possibilitando ao sistema distinguir automaticamente entre modo diurno (ambiente iluminado) e modo noturno (ambiente escuro).

Sempre que o valor lido pelo LDR desce abaixo de um determinado limite, o sistema interpreta que se encontra num ambiente com pouca luz e entra em modo noturno. Este estado é indicado visualmente através de um LED adicional ligado ao pino GP14, que acende apenas quando o sistema está em modo noturno. Caso contrário, o LED permanece apagado, indicando o modo diurno.

Para além da sinalização visual, foi também integrado um buzzer ativo ligado ao pino GP17, que fornece feedback sonoro de acordo com o estado do sistema:

- Em modo standby (sem deteção de movimento), o buzzer emite um som grave e breve, a intervalos regulares, simulando um estado de vigilância passiva.
- Quando o sensor PIR deteta movimento, o sistema entra em modo de gravação e o buzzer emite dois bips agudos curtos, simulando um alerta de atividade.

Este comportamento oferece uma combinação clara de feedback auditivo e visual, permitindo ao utilizador perceber o estado atual do sistema a qualquer momento, mesmo sem acesso direto à interface web. Assim, esta funcionalidade contribui para uma experiência de monitorização mais imersiva e realista, aproximando-se do comportamento de sistemas de vigilância profissionais.

Alteração automática do tema com base na luminosidade (modo claro/escuro)

Foi implementado no sistema um mecanismo de deteção de luz ambiente, utilizando um sensor LDR (Light Dependent Resistor) ligado ao pino GP28 do Raspberry Pi Pico W. Este sensor permite distinguir entre ambientes com e sem iluminação, possibilitando ao sistema alterar automaticamente entre modo diurno e modo noturno.

Sempre que o valor lido pelo LDR desce abaixo de um limite previamente definido (com base em calibração), o sistema considera que se encontra num ambiente escuro e publica a mensagem "NOTURNO" no tópico MQTT camera/modo. Caso o valor esteja acima desse limite, o sistema publica "DIURNO", indicando um ambiente iluminado.

A interface web (index_mqtt_login.html) está ligada a esse tópico via MQTT e responde dinamicamente à mudança de estado:

- Em modo diurno, aplica um tema visual claro (fundo branco e texto escuro);
- Em modo noturno, aplica um tema escuro (fundo escuro e texto claro).

Esta alteração automática melhora significativamente a experiência do utilizador, proporcionando um ambiente visual mais agradável e adaptado à luminosidade do espaço. Além disso, esta funcionalidade reforça o comportamento realista do sistema, simulando o funcionamento de câmaras de vigilância que adaptam os seus modos de operação consoante as condições de luz.



Figura 4 - Modo Noturno



Figura 5 - Modo Diurno

Calibração do Sensor LDR

Para assegurar que o sistema consegue distinguir de forma fiável entre modo diurno e modo noturno, foi realizada a calibração do sensor de luminosidade (LDR). O objetivo deste processo foi determinar os limites reais de leitura do sensor nas diferentes condições de iluminação, de forma a definir um limite eficaz para alternar entre os dois modos de funcionamento.

Para isto foram feitos alguns procedimentos tais como:

Foi executado um script que lia continuamente os valores analógicos do LDR, ligado ao pino GP28 do Raspberry Pi Pico W. Durante o ensaio, foram simuladas 2 condições distintas:

- Luz ambiente normal (ex: luz da sala);
- Escuridão total (sensor tapado).

O código registou os valores mínimos e máximos detetados nessas condições, resultando nos seguintes dados:

- Valor mínimo registado (ambiente escuro): 4016
- Valor máximo registado (luz normal): 61389

A partir destes valores, foi calculado o limite ideal para a mudança de estado:

Limite = (4016 + 61389) / 2 = 32927

Este valor foi então utilizado na função is_noite() do código principal:

```
def is_noite():
    valor = ldr.read_u16()
    print("LDR:", valor)
```

return valor < 32927

Concluímos assim que, com esta calibração, o sistema consegue interpretar com fiabilidade a luminosidade do ambiente e alternar automaticamente entre os modos DIURNO e NOTURNO, adaptando o comportamento do LED indicador e da câmara em conformidade com as condições reais.

Implementação de Funcionalidades MQTT: Keep Alive, Last Will e Retained Message

1. Keep Alive

O parâmetro Keep Alive foi configurado no lado do dispositivo (Raspberry Pi Pico W) com um valor de 60 segundos, o que garante que o broker verifica periodicamente se a ligação está viva.

Implementação:

```
try:
    client = MQTTClient(
        CLIENT_ID,
        MQTT_BROKER,
        user=MQTT_USER,
        password=MQTT_PASS,
        keepalive=60
    )
```

Se a ligação não enviar nenhum sinal ao broker durante esse intervalo, o broker assume que a ligação foi perdida.

2. Last Will and Testament (LWT)

A funcionalidade LWT permite ao broker publicar uma mensagem automaticamente quando o cliente MQTT (dispositivo) perde a ligação de forma inesperada (ex: falha de energia).

Implementação:

Esta mensagem é publicada no tópico camera/erro.

```
client.set_last_will(b"camera/erro", b"ERRO: Corte de Energia Inesperado, Camara Desligada!", retain=False)
```

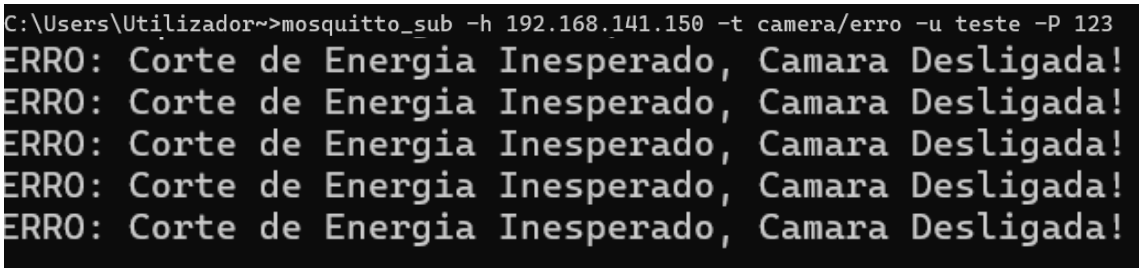
No front-end (index_mqtt_login.html), a mensagem é captada e apresentada num alerta visual no topo da página:

```
if (topic === "camera/erro") {
  const alerta = document.getElementById("erro-alerta");
  alerta.style.display = "block";

  fetch("https://ntfy.sh/erro-camera-desligada", {
    method: "POST",
    body: "⚠️ A câmara desligou-se inesperadamente (ex: corte de energia)."
  });

  setTimeout(() => {
    alerta.style.display = "none";
  }, 10000);
}
```

Como resultado, o utilizador vê um aviso visual: "⚠️ Corte de Energia Inesperado, Câmara Desligada!" e é também enviada uma notificação via ntfy.sh.



3. Retained Messages

As mensagens retidas (retained) são usadas para garantir que, quando um novo cliente se liga e subscreve um tópico, recebe o último estado conhecido sem ter de esperar por uma nova publicação.

Exemplo de utilização:

- client.publish(TOPIC_STATUS, b"ON", retain=True)
- client.publish(TOPIC_SOM, estado, retain=True)
- client.publish(TOPIC_MODO, b"NOTURNO", retain=True)

Estas mensagens são marcadas como retain=True, para que o broker guarde o último estado da câmara, do som e do modo.

No front-end, o cliente recebe imediatamente esses estados assim que se liga e os apresenta corretamente no painel.

Concluímos assim que a implementação destas três funcionalidades melhora significativamente a robustez e fiabilidade do sistema:

- O keep alive monitoriza a ligação.

- O LWT notifica falhas inesperadas.
- As mensagens retidas mantêm o front-end sincronizado mesmo após reconexões.



Figura 6 - Last Will

Integração com API Externa para Notificações em Tempo Real (ntfy.sh)

Foi integrada uma API externa de notificações push, o serviço ntfy.sh. Esta ferramenta permite receber notificações em tempo real sempre que há uma alteração relevante no estado do sistema, como a deteção de movimento, alteração do modo de operação (diurno/noturno) ou o controlo manual da câmara através da interface web.

A cada alteração de estado nos principais tópicos MQTT ou interação com botões da interface, o sistema executa um pedido HTTP do tipo POST para um tópico público no serviço ntfy.sh, enviando uma mensagem simples.

Exemplos de notificações enviadas:

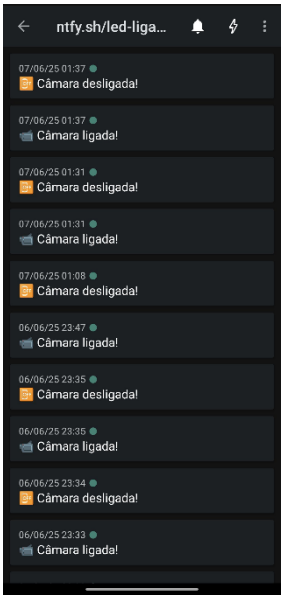
- Quando o modo passa para noturno:

```
fetch("https://ntfy.sh/noturno-ligado-1717", {
method: "POST",
body: "🌓 Modo alterado para: NOTURNO"
});
```



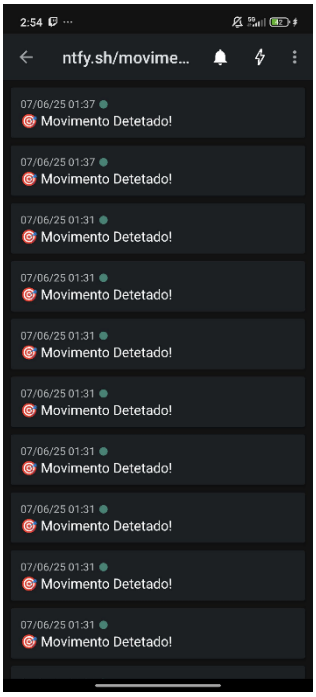
- Quando o botão “Ligar” é pressionado:

```
fetch("https://ntfy.sh/led-ligado-1717", {
method: "POST",
body: "📹 Câmara ligada!"
});
```



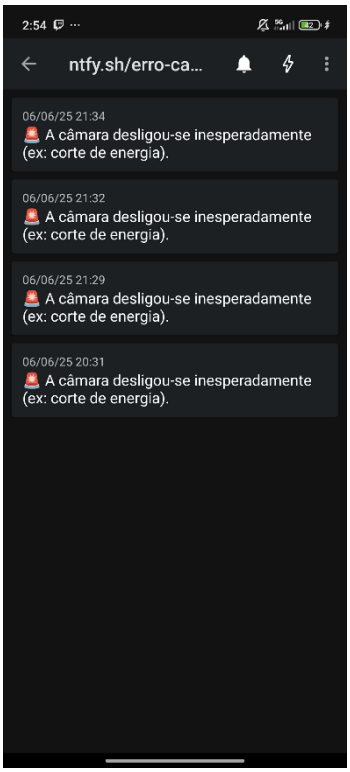
- Quando há deteção de movimento:

```
fetch("https://ntfy.sh/movimento-detetado-1717", {
method: "POST",
body: "🎯 Movimento Detetado!"
});
```



- Quando ocorre uma falha inesperada (ex: corte de energia):

```
fetch("https://ntfy.sh/erro-camera-desligada", {
method: "POST",
body: "🚨 A câmara desligou-se inesperadamente."
});
```



Concluímos assim, a integração com a API ntfy.sh que amplia a eficácia do sistema, proporcionando uma camada adicional de interatividade e vigilância remota. O utilizador é imediatamente notificado sobre qualquer evento relevante, tornando o sistema mais inteligente, prático e fiável no contexto de segurança e monitorização.

The solution should also include a provisioning mechanism to add, remove, and replace devices.

Após diversas e variadas tentativas sem sucesso, o nosso trabalho neste ponto não ficou totalmente completo, porém a nossa melhor tentativa vai ser aqui demonstrada.

Um dos requisitos do projeto era incluir um mecanismo de provisionamento que permitisse adicionar, remover e substituir dispositivos, nomeadamente no que diz respeito à gestão de redes Wi-Fi e respetivas credenciais.

Apesar de diversas tentativas, não foi possível concluir integralmente este ponto. Foram desenvolvidas e testadas várias abordagens, incluindo:

- A possibilidade de guardar múltiplos SSIDs num ficheiro config.json, permitindo ao dispositivo tentar ligar-se sequencialmente a redes conhecidas;
- A tentativa de eliminar automaticamente o ficheiro config.json caso a ligação Wi-Fi ou o MQTT falhasse, de forma a forçar o recomeço do processo de configuração;
- A ideia de apresentar um portal de configuração para inserção manual de novos SSIDs e passwords, caso a rede guardada não estivesse disponível.

No entanto, estas abordagens revelaram-se instáveis em contexto real, embora fosse possível guardar o primeiro SSID com sucesso, a substituição por um novo SSID nem sempre funcionava corretamente. A ligação falhava e o dispositivo nem sempre ativava o modo de configuração como esperado.

Com isto a solução final que adotámos foi optar por um sistema simplificado, em que:

- Apenas um SSID e password são guardados no ficheiro config.json;
- O ficheiro é gerado manualmente ou através de um portal de configuração Wi-Fi, acessível via hotspot local (PicoConfig);
- O sistema tenta ligar-se à rede guardada no arranque. Caso não consiga, entra automaticamente em modo de configuração, permitindo a inserção de um novo SSID/password que substitui o anterior.

Embora este mecanismo não permita gerir múltiplas redes nem guardar um histórico de SSIDs, garante uma experiência de utilização simples e fiável, adequada ao contexto do projeto. O dispositivo é assim preparado para funcionar com uma única rede principal de cada vez, podendo ser reconfigurado sempre que necessário.

Conclusão

O projeto desenvolvido permitiu criar um sistema de câmara de vigilância IoT funcional, baseado em hardware de baixo custo e consumo reduzido. Foram integrados sensores e atuadores para deteção de movimento, variação de luminosidade e controlo remoto via interface web, com comunicação assente no protocolo MQTT.

Embora nem todos os objetivos tenham sido plenamente atingidos (nomeadamente no que diz respeito ao "Provisioning"), foi possível implementar uma solução estável e prática, com autenticação, notificações remotas e alteração automática entre modos de funcionamento. O sistema cumpre os principais requisitos funcionais e apresenta potencial para evolução futura.